

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I Gurusurya G 192311332 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Gurusurya

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. MD5 Password Hashing

Weaknesses of MD5:

- MD5 produces only a 128-bit hash, which is relatively weak by modern security standards.
- It is extremely fast, allowing attackers to perform millions or billions of password guesses quickly using GPUs.
- MD5 has known collision vulnerabilities, meaning different inputs can produce the same hash.
- If passwords are stored without unique salts, attackers can use rainbow tables to crack many passwords efficiently.
- MD5 is not designed specifically for password storage.

More secure approach:

Use a modern password-hashing algorithm such as PBKDF2-HMAC-SHA-256. Each password should have a unique random salt and a sufficiently large iteration count.

Example process:

Password + Unique Salt → PBKDF2-HMAC-SHA-256 → Stored Hash

SHA-256 by itself is much stronger than MD5 for general hashing, but for passwords, PBKDF2 with SHA-256 is preferable because its computational cost can be increased to slow down brute-force attacks.

2. Digital Signature Implementation in Banking

Unauthorized transactions being approved may indicate that the digital signature system is not correctly verifying the sender, signature, transaction data, or private key.

Possible flaws:

- Private keys may have been stolen or poorly protected.
- The application may verify only that a signature is valid without checking the correct user's identity.
- The transaction amount or account number may not be included in the signed data.
- Weak or outdated signature algorithms may be used.
- Certificate validation may be incorrectly implemented.
- Replay attacks may allow a previously valid signed transaction to be submitted again.
- Poor key management or improper random-number generation can compromise signatures.

Recommended improvements:

- Use established standards such as RSA-PSS or ECDSA/EdDSA with appropriate security parameters.
- Protect private keys using secure key storage, preferably HSMs or equivalent protected key-management systems.
- Sign the complete transaction details, including account, amount, recipient, timestamp, and transaction ID.
- Validate the user's certificate and certificate chain.
- Use timestamps, nonces, and unique transaction IDs to prevent replay.
- Implement strong key generation, rotation, revocation, and access controls.

- Log and audit signature verification failures and suspicious transactions.

3. Kerberos Replay Attack

A Kerberos replay attack occurs when an attacker captures a previously transmitted authentication message, such as an authenticator, and sends it again to gain access.

How it can occur:

1. A legitimate client authenticates with the Kerberos Key Distribution Center (KDC).
2. The client sends an authenticator to a server.
3. An attacker captures this network traffic.
4. If the server does not properly check timestamps or detect duplicate authenticators, the attacker can replay the captured message.
5. The server may incorrectly treat the replayed message as a valid authentication request.

Enhancements:

- Use Kerberos timestamps and enforce a small allowable clock-skew window.
- Servers should maintain replay caches to detect previously used authenticators.
- Synchronize clocks using secure time synchronization mechanisms.
- Use strong encryption and secure Kerberos configurations.
- Protect Ticket Granting Tickets (TGTs) and service tickets.
- Use short ticket lifetimes where appropriate.
- Ensure the KDC and participating systems are securely configured and patched.

Thus, timestamps + replay detection + secure ticket management significantly reduce the risk of Kerberos replay attacks.

4. HMAC for Message Authentication

A simple hash such as:

Hash(Message)

only verifies the message's hash value. If an attacker modifies the message, they can calculate a new hash for the modified message and replace the original hash. Since there is no secret involved, the receiver cannot determine who generated the hash.

Solution: HMAC

HMAC combines a secret key with a cryptographic hash function.

For example:

HMAC-SHA-256(Secret Key, Message) → Authentication Tag

The sender sends the message together with the HMAC. The receiver uses the same secret key to calculate the HMAC again.

- If both HMAC values match → the message is authentic and has not been modified.
- If they do not match → the message has been modified or the sender does not possess the correct key.

Recommended implementation:

- Use HMAC-SHA-256 or HMAC-SHA-384/512.
- Generate strong, random secret keys.
- Store keys securely.

- Use different keys for different systems or purposes when appropriate.
- Compare HMAC values using a constant-time comparison method.
- Use unique message identifiers/nonces where replay protection is required.

HMAC therefore provides both message integrity and authentication, unlike an ordinary unkeyed hash.

5. X.509 Certificate Trust Validation

Frequent X.509 trust errors can result from problems with certificates, certificate chains, or the trust store.

Possible issues:

- The certificate has expired or is not yet valid.
- The certificate's hostname does not match the server name.
- The certificate was issued by an untrusted Certificate Authority (CA).
- An intermediate CA certificate is missing.
- The certificate chain is incorrectly configured.
- A certificate has been revoked.
- The client's trust store contains outdated or missing root CA certificates.
- Incorrect system date/time can cause certificates to appear invalid.
- The certificate uses an outdated or insecure algorithm.

Recommended solutions

A proper validation process should be:

Server Certificate → Intermediate CA → Trusted Root CA

Organizations should:

- Use certificates issued by trusted CAs.
- Install the complete certificate chain, including required intermediate certificates.
- Maintain an updated trust store.
- Check certificate expiration and renew certificates before expiry.
- Verify the hostname using the certificate's Subject Alternative Name (SAN).
- Check certificate revocation using appropriate mechanisms such as CRL or OCSP, where applicable.
- Use modern TLS versions and secure cryptographic algorithms.
- Maintain accurate system clocks.
- Automate certificate monitoring and renewal.

Overall, proper X.509 management requires correct certificate issuance, complete chain configuration, trusted root management, hostname verification, expiration monitoring, and revocation checking.